

RAG

TIPTAP

BULLMQ

STRIPE

PGVECTOR

Plateforme SaaS

RAG pour

Generation de

Rapports

Academiques

Architecture complete, detaillee jusqu'au niveau fichier et module, concue pour le vibecoding avec Claude Code.

Stack Next.js 14, PostgreSQL + pgvector, Claude Sonnet, et pipeline RAG optimise pour 25 utilisateurs en phase test.

Stack : Next.js 14 + TypeScript + Tailwind + shadcn/ui

Base de donnees : PostgreSQL (Neon) + pgvector

LLM : Claude Sonnet (Anthropic API)

Phase : Test ~25 utilisateurs

1. Vue d'ensemble et stack technique

1.1 Contexte et objectif du produit

1.2 Stack technique definitif

1.3 Architecture systeme

2. Arborescence projet (Next.js App Router)

2.1 Structure /lib (logique metier)

2.2 Structure /components (composants UI)

3. Schema de donnees complet (Drizzle ORM)

3.1 Enumerations

3.2 Tables principales

3.3 Details des tables

4. Pipeline RAG (coeur du produit)

4.1 Phase 1 : Indexation offline du corpus

4.2 Phase 2 : Generation du sommaire

4.3 Phase 3 : Generation du rapport complet

5. Pipeline detaille par etape

5.1 Etape A : Onboarding projet

5.2 Etape B : Generation sommaire

5.3 Etape C : Edition iterative du sommaire

5.4 Etape D : Generation rapport complet

5.5 Etape E : Editeur split-view

6. Gestion des quotas et Stripe Billing

6.1 Middleware de verification des quotas

6.2 Tiers d'abonnement suggeres

6.3 Webhook Stripe

7. Export final (DOCX et PDF)

7.1 Export DOCX

7.2 Export PDF

3

3

3

4

5

6

7

7

7

8

8

9

9

10

10

11

11

11

11

11

12

12

13

13

13

14

8. Points d'attention et securite **14**

8.1 Confidentialite et isolation des donnees 14

8.2 Suivi des couts API 14

8.3 Corpus de reference : conformite legale 14

8.4 Rate limiting et securite API 14

9. Ordre de vibecoding recommande **15**

9.1 Estimation de couts (25 utilisateurs en test) 15

1. Vue d'ensemble et stack technique

1.1 Contexte et objectif du produit

Ce document presente l'architecture technique complete d'une plateforme SaaS destinee a la generation automatisee de rapports academiques (PFE et stages). Le coeur du produit repose sur un pipeline RAG (Retrieval-Augmented Generation) qui utilise un corpus d'anciens rapports de reference pour generer des sommaires structures, puis des rapports complets section par section. L'editeur split-view, base sur TipTap, permet a l'utilisateur de selectionner un passage specifique du rapport et de le modifier via Claude avec une instruction ciblee, le tout dans une interface avant/apres avec diff. L'architecture est concue pour une phase test a environ 25 utilisateurs, avec un cout mensuel quasi nul a 20-40 dollars hors consommation API Claude.

Le flux produit complet suit un parcours en sept etapes : inscription et abonnement via Stripe, creation de projet avec brief et upload de fichiers, generation du sommaire par RAG, edition iterative du sommaire par l'utilisateur, validation du sommaire, generation du rapport complet en arriere-plan via queue BullMQ, et enfin edition fine dans l'editeur split-view suivie de l'export final en DOCX ou PDF. Chaque etape est detaillee dans les sections suivantes avec les decisions techniques, le code de reference et les points d'attention specifiques.

1.2 Stack technique definitif

Le choix de chaque technologie de la stack est motive par une contrainte de vibecoding rapide (iteration frequente, faible overhead operationnel) et de budget minimal en phase test. Voici la justification detaillee pour chaque couche de l'architecture.

Couche	Technologie	Justification
Frontend	Next.js 14 App Router + TypeScript + Tailwind + shadcn/ui	SSR + API routes integrees, vibecoding rapide, ecosysteme riche
Editeur riche	TipTap v2	Selections structurees (nodes, ranges) via ProseMirror, extensible, patch cible
Base de donnees	PostgreSQL (Neon) + pgvector	RAG natif avec similarity search, RLS pour isolation user, serverless et gratuit en phase test
Cache/Queue	Upstash Redis + BullMQ	Jobs async pour generation longue (1-3 min), serverless pay-per-use
Stockage fichiers	Cloudflare R2	S3-compatible, pas d'egress fees, moins cher que S3 a faible volume
Authentification	Clerk	Integration rapide, gestion sessions/JWT, UI pre-construite

Paielement	Stripe Billing	Gestion des tiers, quotas, factures, webhook fiable
LLM	Anthropic API (Claude Sonnet)	Generation de texte long, edition ciblee, bonne gestion du contexte
Embeddings	Voyage AI (voyage-3)	Recommande par Anthropic, 1024 dimensions, bon rapport qualite/prix
Export DOCX	docx (npm)	Generation native Word avec styles academiques (Titre 1/2/3, TOC)
Export PDF	Puppeteer	Rendu HTML vers PDF fidele pour mise en page academique
Hebergement	Vercel + Neon + Upstash + R2	Cout quasi nul a 25 users, deploy instantane, scaling automatique
Monitoring	Sentry + Vercel Analytics	Detection erreurs et perf des le jour 1 de la production

Tableau 1 : Stack technique definitif avec justifications

1.3 Architecture systeme

L'architecture suit un modele en couches distinctes : le frontend Next.js gere la UI et les interactions utilisateur (TipTap, shadcn/ui), les API routes Next.js implementent la logique metier et les endpoints REST, les workers BullMQ traitent les jobs longs (generation sommaire et rapport), et la couche data comprend PostgreSQL (Neon) avec l'extension pgvector pour le stockage vectoriel, Redis (Upstash) pour la queue et le cache, et Cloudflare R2 pour le stockage des fichiers uploades. Les services externes incluent Claude via Anthropic API pour la generation, Voyage AI pour les embeddings, et Stripe pour la facturation.

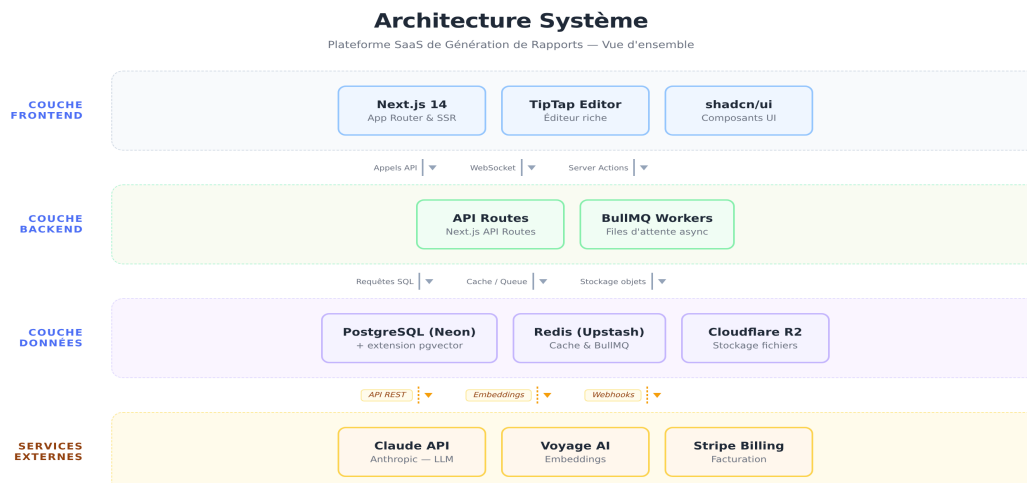


Figure 1 : Architecture système en couches avec flux de données

2. Arborescence projet (Next.js App Router)

L'arborescence suit les conventions du Next.js 14 App Router avec un routage basé sur les dossiers. Les routes sont organisées en trois groupes principaux : (marketing) pour les pages publiques (landing, pricing), (auth) pour l'inscription et la connexion, et (app) pour le tableau de bord et les fonctionnalités principales. Le layout (app) inclut la sidebar de navigation et le widget de quota. Chaque route API est placée sous /api avec une structure CRUD standard.

Chemin	Role
/app/(marketing)/page.tsx	Landing page publique
/app/(marketing)/pricing/page.tsx	Page tarifs et plans d'abonnement
/app/(auth)/sign-in/	Formulaire de connexion (Clerk)
/app/(auth)/sign-up/	Formulaire d'inscription (Clerk)
/app/(app)/layout.tsx	Sidebar + Quota widget (layout partage)
/app/(app)/dashboard/	Tableau de bord utilisateur
/app/(app)/projects/new/	Formulaire brief + upload fichiers
/app/(app)/projects/[id]/	Vue projet / statut des jobs

/app/(app)/projects/[id]/sommaire/	Editeur sommaire (etape 1)
/app/(app)/projects/[id]/editor/	Split-view rapport (etape 2)
/app/(app)/settings/billing/	Gestion abonnement Stripe
/app/api/projects/	POST create, GET list
/app/api/projects/[id]/	GET, PATCH, DELETE par projet
/app/api/projects/[id]/files/	Upload fichiers vers R2
/app/api/generation/sommaire/	POST vers queue BullMQ
/app/api/generation/report/	POST vers queue BullMQ
/app/api/generation/section-edit/	POST edition ciblee (sync)
/app/api/generation/jobs/[jobId]/	GET statut job (polling/SSE)
/app/api/rag/index-reference/	Admin: indexer ancien rapport
/app/api/webhooks/stripe/	Webhook Stripe Billing
/app/api/export/docx/[projectId]/	Export DOCX
/app/api/export/pdf/[projectId]/	Export PDF

Tableau 2 : Arborescence des routes Next.js App Router

2.1 Structure /lib (logique metier)

Le dossier /lib contient toute la logique metier isolee des routes. Chaque sous-module est organise par domaine fonctionnel : db/ pour le schema Drizzle ORM et les queries, llm/ pour le client Anthropic et les prompts structures, rag/ pour le chunking et le retrieval vectoriel, queue/ pour BullMQ et les workers, quota/ pour le middleware de verification des quotas, export/ pour la generation DOCX/PDF, et storage/ pour le client R2. Cette separation stricte permet de tester chaque module independamment et de migrer vers NestJS si necessaire sans reecriture complete.

Module	Fichiers cles	Responsabilite
db/	schema.ts, client.ts, queries/	Schema Drizzle ORM, connexion Neon, queries typees
llm/	anthropic-client.ts, prompts/	Client Claude, prompts sommaire/section/edit
rag/	chunking.ts, retriever.ts	Decoupage rapports, similarity search pgvector
queue/	client.ts, workers/, jobs.ts	BullMQ setup, workers sommaire/report, enqueue

quota/	check-quota.ts	Middleware verification quotas par tier d'abonnement
export/	docx-builder.ts, pdf-builder.ts	Generation DOCX (styles academiques) et PDF
storage/	r2-client.ts	Client Cloudflare R2 (upload/download/delete)

Tableau 3 : Modules du dossier /lib

2.2 Structure /components (composants UI)

Les composants React sont organisés par domaine fonctionnel. Le module editor/ contient les composants clés de l'éditeur split-view : TipTapEditor.tsx pour l'éditeur riche en lecture, SelectionToolbar.tsx pour le popup flottant au survol/sélection, DiffPanel.tsx pour le panneau droite affichant le diff avant/apres, et SommaireOutline.tsx pour l'arborescence editable du sommaire. Le module upload/ contient FileDropzone.tsx pour le glisser-déposer de fichiers, et billing/ contient QuotaWidget.tsx et PricingTable.tsx pour l'affichage des quotas et des plans tarifaires.

3. Schema de donnees complet (Drizzle ORM)

Le schema de données est défini avec Drizzle ORM et utilise PostgreSQL avec l'extension pgvector pour le stockage des embeddings. Chaque table métier inclut des enums PostgreSQL pour les statuts et les types. L'isolation des données par utilisateur est assurée par Row Level Security (RLS) dès le départ, avec une policy `user_id = current_setting('app.current_user_id')::uuid` sur chaque table. Neon supporte RLS nativement, ce qui évite tout risque de fuite de données entre utilisateurs. Les rapports PFE et de stages contiennent souvent des informations sensibles d'entreprises, ce qui rend cette isolation stricte indispensable dès la première migration.

3.1 Enumerations

Trois enumerations définissent les statuts principaux du système : `subscription_tier` pour les niveaux d'abonnement (free, starter, pro), `project_status` pour le cycle de vie d'un projet (draft, generating_sommaire, sommaire_ready, generating_report, report_ready, archived), et `job_status` pour l'état des jobs de génération (queued, processing, completed, failed). Ces enums sont définis au niveau PostgreSQL via pgEnum de Drizzle pour garantir l'intégrité référentielle en base et non uniquement au niveau applicatif.

Enum	Valeurs	Usage
subscription_tier	free, starter, pro	Niveau d'abonnement utilisateur, définit les quotas
project_status	draft, generating_sommaire, sommaire_ready, generating_report, report_ready, archived	Cycle de vie du projet, pilote le workflow de génération

job_status	queued, processing, completed, failed	Etat des jobs BullMQ, affiche la progress bar
------------	---------------------------------------	---

Tableau 4 : Enumerations PostgreSQL

3.2 Tables principales

Le schema comprend neuf tables principales couvrant l'ensemble du domaine metier. La table users stocke les informations de l'utilisateur avec ses quotas de generation et de stockage. La table projects represente un projet de rapport avec son type (PFE ou stage), sa filiere (sector) pour le matching RAG, et son statut. Les uploaded_files contiennent les fichiers de reference uploades avec le texte extrait apres parsing (PDF via pdf-parse, DOCX via mammoth). Le corpus de reference est split en deux tables : reference_reports pour les metadonnees anonymisees, et reference_chunks pour les fragments embeddes avec leurs embeddings vectoriels en 1024 dimensions (voyage-3).

La table sommaire_versions gere le versioning des sommaires avec un champ contentJson stockant l'arborescence complete en JSON, et un flag is_validated indiquant si le sommaire est pret pour la generation du rapport. Les report_sections stockent chaque section du rapport final en HTML avec son ordre d'affichage. La table edit_history trace chaque operation d'edition ciblée avec le texte avant/apres et l'instruction utilisateur, permettant de revenir en arriere. Enfin, generation_jobs suit l'etat de chaque job de generation avec une progress bar (0-100).

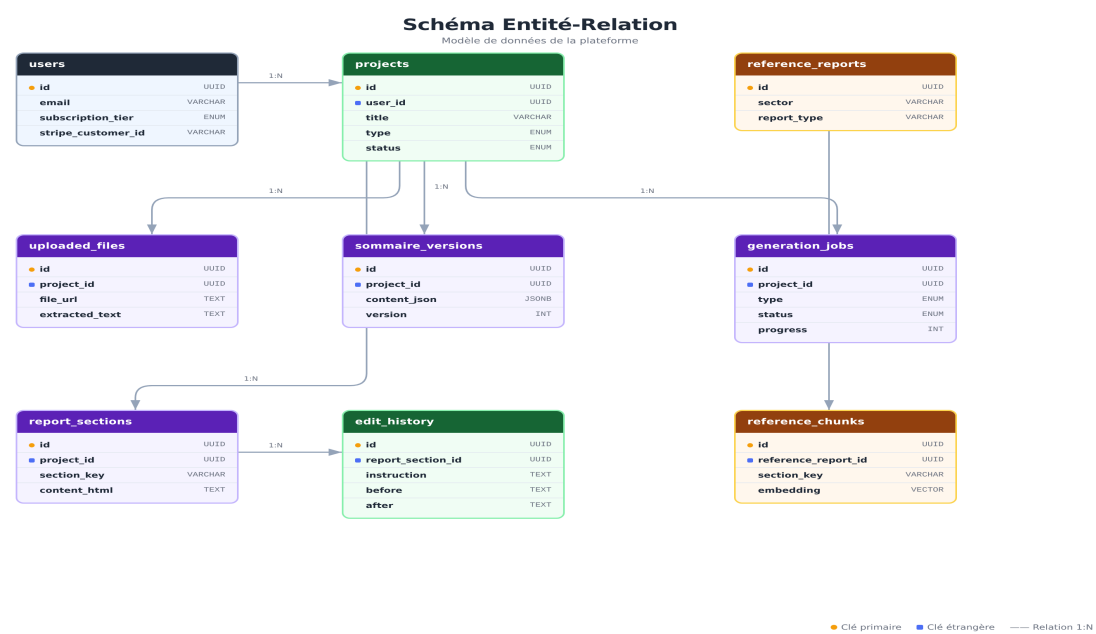


Figure 2 : Schema entite-relation complet du systeme

3.3 Details des tables

Table	Colonnes cles	Notes
-------	---------------	-------

users	id, clerkId, email, subscription_tier, quota_generations_used/limit, quota_storage_mb_used/limit	clerkId unique, quotas selon tier
projects	id, user_id (FK), title, type, sector, status, description	type = 'pfe' ou 'stage', sector pour matching RAG
uploaded_files	id, project_id (FK), file_url, file_name, mime_type, size_bytes, extracted_text	extracted_text rempli apres parsing PDF/DOCX
reference_reports	id, sector, report_type, source_label	source_label anonymise pour conformite legale
reference_chunks	id, reference_report_id (FK), section_key, content, embedding (vector 1024)	vector pgvector voyage-3, section_key = type de section
sommaire_versions	id, project_id (FK), content_json (JSONB), version, is_validated	versioning simple, JSON pour arborescence flexible
report_sections	id, project_id (FK), section_key, title, content_html, order_index	content_html pour rendu TipTap direct
edit_history	id, report_section_id (FK), selected_text, instruction, before_content, after_content, accepted	Audit complet de chaque edition ciblee
generation_jobs	id, project_id (FK), type, status, progress, error_message	type = 'sommaire' ou 'full_report', progress 0-100

Tableau 5 : Details des tables du schema de donnees

4. Pipeline RAG (coeur du produit)

Le pipeline RAG est le coeur differentiant de la plateforme. Il se decompose en trois phases distinctes : l'indexation offline du corpus de reference, la generation du sommaire structure, et la generation du rapport complet section par section. Chaque phase exploite les embeddings voyage-3 stockes dans pgvector pour retrouver les passages les plus pertinents et les injecter dans les prompts Claude, garantissant ainsi des rapports coherents avec le ton, la structure et les formulations des rapports academiques de reference de la meme filiere.

4.1 Phase 1 : Indexation offline du corpus

L'indexation se fait en une seule fois via un script admin (pas d'UI necessaire en phase test). Le processus est le suivant : les anciens rapports sont chunks par section (introduction, contexte de l'entreprise, methodologie, resultats, conclusion, etc.), chaque chunk est embedde via Voyage AI en un vecteur de 1024 dimensions, puis stocke dans la table reference_chunks avec des metadonnees (section_key, filiere, type de rapport).

L'extraction du texte se fait via pdf-parse pour les PDF et mammoth pour les DOCX. Il est impératif de vérifier le droit d'utiliser ces rapports de référence (consentement des auteurs ou anonymisation complète), car les PFE et stages contiennent souvent des informations confidentielles sur les entreprises d'accueil.

4.2 Phase 2 : Generation du sommaire

Lorsque l'utilisateur valide son brief projet, le backend envoie le brief à Voyage AI pour obtenir un embedding, puis effectue une similarity search dans reference_chunks filtrée par sector et report_type. Les top-K structures de sommaires similaires sont récupérées et injectées dans un prompt structure destiné à Claude. Ce prompt contient le brief de l'étudiant, les exemples de structures de sommaires similaires, et demande une sortie JSON stricte définissant les sections et sous-sections. Le résultat est sauvegardé dans sommaire_versions avec version = 1 et is_validated = false.

4.3 Phase 3 : Generation du rapport complet

Après validation du sommaire par l'utilisateur, la génération du rapport complet est déclenchée via un job BullMQ (car la génération prend 1 à 3 minutes). Le worker traite chaque section du sommaire validé séparément, et non en un seul prompt. Pour chaque section, il effectue un retrieval dans reference_chunks filtré par section_key similaire, récupère les top-5 passages de référence, puis construit un prompt Claude avec le contexte du projet, les sections précédentes déjà générées (pour la cohérence), et les exemples de référence. Cette approche section par section permet une reprise sur erreur, une barre de progression réelle, et la régénération ciblée d'une section sans tout refaire.

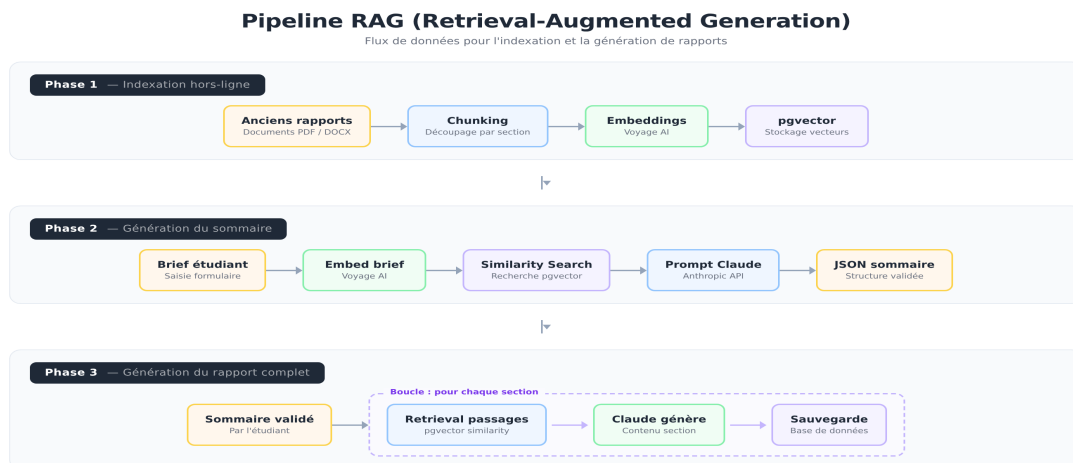


Figure 3 : Pipeline RAG complet (indexation, generation sommaire, generation rapport)

5. Pipeline detaille par etape

5.1 Etape A : Onboarding projet

L'onboarding commence par la creation du projet via `POST /api/projects`, qui cree un enregistrement en statut draft dans la table projects. Ensuite, l'utilisateur uploades ses fichiers (brief, supports, captures d'ecran) via `POST /api/projects/[id]/files`. Chaque fichier est stocke dans Cloudflare R2, puis le texte est extrait : PDF via pdf-parse, DOCX via mammoth, images via Claude Vision si un contexte visuel est necessaire. Avant l'upload, le middleware check-quota.ts verifie que la taille totale et le nombre de fichiers sont compatibles avec le tier d'abonnement de l'utilisateur.

5.2 Etape B : Generation sommaire

`POST /api/generation/sommaire` envoie immediatement un jobId (HTTP 202) et enqueue le job dans BullMQ. Le worker (sommaire-worker.ts) execute la logique RAG decrite au chapitre precedent. Le frontend fait du polling sur `GET /api/generation/jobs/[jobId]` ou utilise SSE (Server-Sent Events) via ReadableStream de Next.js pour un suivi en temps reel de la progression. Une fois termine, le resultat est sauvegarde dans `sommaire_versions` et `generation_jobs.status` passe a completed. L'utilisateur est redirige vers l'editeur de sommaire.

5.3 Etape C : Edition iterative du sommaire

L'interface SommaireOutline.tsx presente une arborescence editable avec drag-drop pour le reordonnancement, ajout/suppression de sections, et edition inline des titres. Chaque modification cree une nouvelle version dans `sommaire_versions` (version incrementale). Le bouton "Valider" passe `is_validated` a true et declenche l'etape D. Le versioning est simple (pas de diff complexe), ce qui suffit pour une phase test.

5.4 Etape D : Generation rapport complet

`POST /api/generation/report` enqueue un second job BullMQ. Le worker (report-worker.ts) itere sur chaque section du sommaire valide. Pour chaque section, il effectue un retrieval dans `reference_chunks`, genere le contenu via Claude, sauvegarde dans `report_sections`, et met a jour la progress bar du job (0 a 100%). L'approche section par section garantit la coherence contextuelle car chaque prompt inclut les sections precedentes deja generees. En cas d'erreur sur une section, le worker peut reprendre sans regenerer les sections deja terminees.

5.5 Etape E : Editeur split-view

L'editeur split-view est la partie la plus differenciante du produit. A gauche, TipTapEditor.tsx affiche le rapport en mode lecture avec les `report_sections` chargees en HTML. L'evenement `onSelectionUpdate` de TipTap capture la selection structuree : `{ from, to, selectedText, sectionId }`. SelectionToolbar.tsx affiche un

popup flottant "Modifier ce passage" qui ouvre le panneau droit.

Le panneau droit (DiffPanel.tsx) presente un champ libre pour l'instruction utilisateur ("rends ce paragraphe plus formel", "ajoute un exemple concret", "raccourcis"), puis envoie un POST /api/generation/section-edit de maniere synchrone (pas de queue, car c'est rapide). Le backend reconstruit le contexte (section entiere + passage selectionne + instruction), appelle Claude avec un prompt cible "modifie uniquement ce passage, garde la coherence avec le reste", et retourne { before: selectedText, after: result.newText }.

Le diff s'affiche avec le texte original barre en rouge et le texte propose en vert (via diff-match-patch pour un diff mot-a-mot). Trois actions sont possibles : Accepter (patch applique dans TipTap via editor.chain().focus().insertContentAt({from, to}, newContent).run(), ce qui remplace le passage selectionne sans re-render complet du document), Rejeter (retour a l'original), ou Regenerer (nouvel appel Claude avec la meme instruction). Chaque edition est tracee dans edit_history.

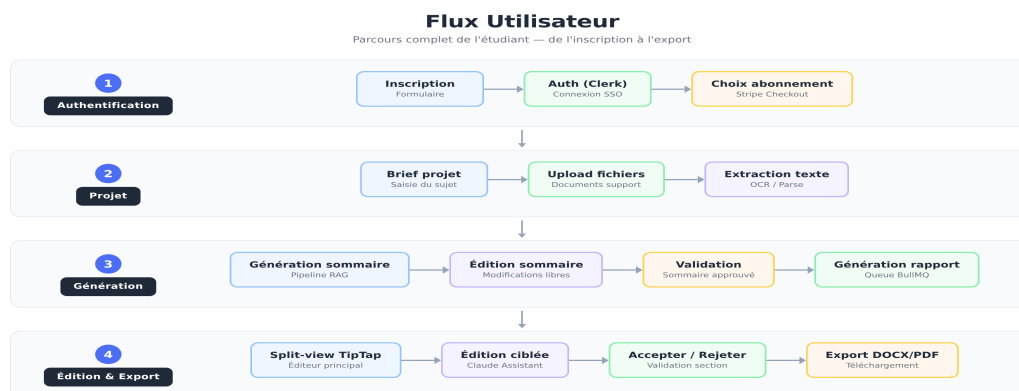


Figure 4 : Flux utilisateur complet de l'inscription à l'export

6. Gestion des quotas et Stripe Billing

6.1 Middleware de verification des quotas

Le fichier lib/quota/check-quota.ts exporte une fonction checkQuota(userId, action, size?) qui est appelée au debut de chaque route sensible (generation/*, files). La fonction verifie que l'utilisateur n'a pas depasse sa limite de generations ou de stockage selon son tier d'abonnement. Le quota est incremente apres succes uniquement (pas avant) pour ne pas penaliser un echec. En cas de depassement, une erreur QuotaExceededError est levee et interceptee par le handler d'erreur global, qui retourne un HTTP 429 avec un message invitant l'utilisateur a upgrader son abonnement.

```
// lib/quota/check-quota.ts
export async function checkQuota(userId, action, size?) {
  const user = await getUserWithQuota(userId);
  if (action === 'generation' && user.quotaGenerationsUsed >= user.quotaGenerationsLimit)
    throw new QuotaExceededError('generation_limit_reached');
  if (action === 'upload' && (user.quotaStorageMbUsed + size) > user.quotaStorageMbLimit)
    throw new QuotaExceededError('storage_limit_reached');
}
```

6.2 Tiers d'abonnement suggeres

Trois tiers sont proposes pour la phase test, calibres pour 25 utilisateurs. Le tier free permet de decouvrir le produit avec une generation et 20 Mo de stockage. Le tier starter offre 3 generations et 100 Mo de stockage a un prix abordable. Le tier pro enleve la limite de generations ciblees et offre 500 Mo de stockage, adapte aux utilisateurs intensifs. Les prix sont exprimes en DT (dinars tunisiens) mais Stripe gere la conversion automatique.

Tier	Prix	Generations/mois	Stockage	Editions ciblees
Free	0 DT	1	20 Mo	10
Starter	15 DT/mois	3	100 Mo	Illimite
Pro	35 DT/mois	10	500 Mo	Illimite

Tableau 6 : Tiers d'abonnement proposes

6.3 Webhook Stripe

Le webhook stripe/route.ts ecoute les evenements Stripe (customer.subscription.updated, invoice.paid, etc.) pour synchroniser le tier d'abonnement dans la table users et reset les quotas a chaque cycle de facturation. Le secret de verification du webhook est stocke dans les variables d'environnement Vercel et jamais expose cote client.

7. Export final (DOCX et PDF)

L'export est la derniere etape du flux utilisateur. Deux formats sont proposes : DOCX pour les utilisateurs qui souhaitent modifier le rapport dans Word ou WPS Office, et PDF pour une version prete a imprimer ou a soumettre. Les deux formats utilisent les report_sections trieess par order_index comme source de donnees.

7.1 Export DOCX

Le module lib/export/docx-builder.ts itere sur les report_sections trieess par order_index et construit un fichier .docx via la librairie docx (npm). Les styles academiques sont appliques automatiquement : Titre 1 pour les sections principales, Titre 2 pour les sous-sections, Titre 3 pour les sous-sous-sections, avec une numerotation

automatique. Une table des matieres auto-generee via le champ Word TOC est incluse en debut de document. Le rendu est fidele aux standards academiques PFE/stages tunisiens.

7.2 Export PDF

Le module lib/export/pdf-builder.ts utilise Puppeteer pour convertir un rendu HTML/CSS en PDF. Cette approche permet un controle precis sur la mise en page (marges, en-tetes, pieds de page, numerotation) et un rendu visuellement fidele a un rapport academique type LaTeX, sans necessite un compilateur LaTeX.

L'alternative est la conversion DOCX vers PDF via LibreOffice headless, mais le rendu Puppeteer est plus fiable pour les mises en page complexes.

8. Points d'attention et securite

8.1 Confidentialite et isolation des donnees

Les rapports PFE et de stages contiennent souvent des informations confidentielles sur les entreprises d'accueil (donnees financieres, strategies, projets en cours). L'isolation stricte par utilisateur est assuree par Row Level Security (RLS) PostgreSQL des la premiere migration, et non pas uniquement par un WHERE user_id au niveau applicatif. Cela signifie que meme si un bug dans le code oublie le filtre user_id, la base de donnees bloquera l'accès aux donnees d'un autre utilisateur. Neon supporte RLS nativement en serverless.

8.2 Suivi des couts API

Chaque appel Claude est logge avec les tokens in/out par utilisateur dans une table api_usage_logs (non detaillee dans le schema principal pour simplicite). Cela permet de calculer le cout reel par utilisateur et d'anticiper la marge par tier d'abonnement. En phase test, cette donnee est essentielle pour calibrer les prix avant le lancement public.

8.3 Corpus de reference : conformite legale

L'utilisation d'anciens rapports comme corpus de reference necessite le consentement explicite des auteurs ou une anonymisation complete. Ce point est particulierement sensible dans le contexte d'un PFE sur l'audit, ou les rapports contiennent des donnees d'entreprises confidentielles. Il est recommande de ne stocker que les structures et formulations types, pas les donnees specifiques aux entreprises, et de mettre en place une politique de retention/forgetting.

8.4 Rate limiting et securite API

Un rate limiting est applique sur les endpoints de generation via un middleware qui verifie le nombre de requetes par minute et par utilisateur. Les cles API (Anthropic, Voyage AI, Stripe) sont stockees exclusivement dans les variables d'environnement Vercel et ne sont jamais exposees cote client. Le CORS est

configure pour n'autoriser que le domaine de l'application. Les webhooks Stripe utilisent la verification de signature pour se proteger contre les requetes non autorisees.

9. Ordre de vibecoding recommande

L'ordre de developpement est optimise pour le vibecoding avec Claude Code : chaque etape est independante et testable, ce qui permet de valider rapidement chaque brique avant de passer a la suivante. L'etape la plus delicate et la plus longue est l'editeur TipTap + split-view, il est recommande de prevoir le plus de temps pour cette partie.

Phase	Tache	Estimation	Dependances
1	Setup Next.js 14 + Drizzle ORM + Neon + RLS de base + Clerk Auth	1-2 jours	Aucune
2	CRUD projets + upload fichiers vers R2 + extraction texte	1-2 jours	Phase 1
3	Indexation manuelle du corpus de reference + pgvector	1 jour	Phase 2
4	Pipeline generation sommaire (sync d'abord, puis BullMQ)	2-3 jours	Phases 2-3
5	UI sommaire editable (drag-drop, versioning)	1-2 jours	Phase 4
6	Pipeline generation rapport complet (queue + progress)	2-3 jours	Phase 5
7	Editeur TipTap + split-view + edition ciblee	3-5 jours	Phase 6
8	Export DOCX/PDF	1-2 jours	Phase 6
9	Stripe Billing + quotas	1-2 jours	Phase 1
10	Monitoring (Sentry) + polish UI	1 jour	Phases 7-9

Tableau 7 : Roadmap de vibecoding avec estimations

9.1 Estimation de couts (25 utilisateurs en test)

Service	Cout mensuel estime	Notes
Vercel (Hobby)	0 \$	Gratuit pour projets personnels
Neon (Free tier)	0 \$	0.5 Go stockage, 100h compute

Upstash Redis	0-5 \$	Pay-per-use, negligeable a 25 users
Cloudflare R2	0-2 \$	10 Go gratuit, pas d'egress fees
Anthropic Claude API	10-30 \$	Variable selon usage, loguer par user
Voyage AI Embeddings	2-5 \$	Indexation initiale + requetes
Stripe	0 \$	Pas de frais tant que pas de transactions reelles
Sentry (Free)	0 \$	5 000 events/mois gratuit
Total estime	12-42 \$/mois	Hors API Claude a l'usage reel

Tableau 8 : Estimation des couts mensuels en phase test